Lists in Python

Agenda

- ~~What is Python? Why Python? How to use it?~~
- ~~Python basics~~
- **Lists**
- Tuples
- Dictionary and sets
- Functions
- Classes
- Try-Catch exception

About List in Python

What is List?

- A list in Python is an **ordered collection of items**, a single variable that can hold multiple values.
- You can **store different types of data in the same list**, such as numbers, strings, or even other lists.

Why Lists?

- Lists are fundamental because they let you work with a group of related data as a single unit.
- This is much **more efficient** than creating separate variables for each item.
- For example, instead of storing a student's grades as **grade1** = 85, **grade2** = 92, **grade3** = 78, you can put them all in a list: **grades** = [85, 92, 78].
- This makes it **easy to perform operations** on all the grades at once, like finding the average or sorting them.

How to use Lists?

You can create Lists by placing items inside **square brackets []**, separated by commas.

List declaration

You create a list using square brackets, in which items are separated by commas.

```
fruits = ["apple", "banana", "cherry"]
print(fruits)
Output:
['apple', 'banana', 'cherry']
```


```
print(type(fruits)) #Output type of variable
Output:
<class 'list'>
```

Access List Items

You can access individual items in a list using their index, which starts at 0.

```
fruits = ["apple", "banana", "cherry"]
print(fruits[0])
```

Output:

```
apple
```


```
print(fruits[1])
```

Output:

```
banana
```


```
fruits = ["apple", "banana", "cherry"]
print(fruits[5])
```

Output:

```
???
```

Access List Items (Negative sign)

- What if I have a **very long list**?
- x = [1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12]
- And I want to access the last item of this list, it should be **x[11]** right?
- What if my list contains millions of items then? **x[1000000]**?
- In this case, you can use **negative sign** to count from the back.

```
x = [1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12]
print(x[-1])
Output:
12
```


```
x = [1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12]
print(x[-4])
Output:
????
```

List Slicing

What if you want to access item 4, 5, 6, 7, 8, 9 as another list?

x[start:stop:step]

```
x = [1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12]
print(x[3:9:1])
Output:  [4, 5, 6, 7, 8, 9]
```


```
x = [1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12]
print(x[3:9])
Output:  [4, 5, 6, 7, 8, 9]
```


```
x = [1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12]
print(x[3:9:2])
Output:  [4, 6, 8]
```

Note: The "stop" is exclusive, which means it won't be included in the sliced list

List Slicing

Try...

```
x = [1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12]

print(x[:])
print(x[:3])
print(x[3:])
```

Output:

```
[1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12]
[1, 2, 3]
[4, 5, 6, 7, 8, 9, 10, 11, 12]
```

Quiz Time

Time Limit: 2 mins

```
x = [1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12]
# Hint: x[start:stop:step]


# 1.Print all odd items in this list using slicing
# 2.Print all even items in this list using slicing
```

Quiz Time

SOLUTION

```
x = [1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12]
# Hint: x[start:stop:step]


# 1.Print all odd items in this list using slicing
print(x[::2]) #OUTPUT: [1, 3, 5, 7, 9, 11]
# 2.Print all even items in this list using slicing
print(x[1::2])#OUTPUT:[2, 4, 6, 8, 10, 12]
```

String Slicing

Just like List Slicing, Words in a String can also be sliced to obtain substring!

```
x = "Python"
print(x[0])
print(x[1:4])
Output:
P
yth
```

Quiz Time

Time Limit: 2 mins

```
#Extract all the IDs from combined_id using String slicing
#syntax = departmentID - categoryID - productID
combined_id = "0021-3233-3221"
```

Quiz Time

Solution

```
#Extract all the IDs from combined_id using String slicing
#syntax = departmentID - categoryID - productID
combined_id = "0021-3233-3221"
departmentID = combined_id[:4]
categoryID = combined_id[5:9]
productID = combined_id[10:]

print("Department ID:", departmentID)
print("Category ID:", categoryID)
print("Product ID:", productID)
```

OUTPUT:

```
Department ID: 0021
Category ID: 3233
Product ID: 3221
```

String Splitting

Another way to obtain substrings that are **separated by certain characters** is by using the split() method.

```
combined_id = "0021-3233-3221"
#separate using split at "-"
departmentID, categoryID, productID = combined_id.split('-')
print("Department ID:", departmentID)
print("Category ID:", categoryID)
print("Product ID:", productID)
```

OUTPUT:

```
Department ID: 0021
Category ID: 3233
Product ID: 3221
```

Range Method

The range() method in Python is a built-in function that **generates a sequence of numbers.**
Syntax: range(start, stop, step)

```
list1 = list(range(11)) #Generate 0 to 10
list2 = list(range(3, 8)) #Generate 3 to 7
list3 = list(range(1, 11, 2)) #Generate 1,3,5,7,9
print("List 1:", list1)
print("List 2:", list2)
print("List 3:", list3)

OUTPUT:
List 1: [0, 1, 2, 3, 4, 5, 6, 7, 8, 9, 10]
List 2: [3, 4, 5, 6, 7]
List 3: [1, 3, 5, 7, 9]
```


```
list(range(9,91,9)) #Multiplication table of 9.
OUTPUT: [9, 18, 27, 36, 45, 54, 63, 72, 81, 90]
```

Changing List Items

To change the value of a specific item, you can refer to the index number.

```
x = [1, 2, 3]
x [1] = 5
print(x)
```

OUTPUT:

```
[1, 5, 3]
```

To remove specific item, you can use pop() method.

15

```
y = [1, 2, 3, 4]
y.pop(2)
print(y)
```

OUTPUT:

```
[1, 2, 4]
```

List Append & Extend

The **append() method** adds a **single element** to the end of a list.
The **extend() method** adds all the elements of an iterable (like another list) to the end of the current list.

```
names = ["William", "John"]
names.append("Alice")
print(names)
```

OUTPUT:

```
['William', 'John', 'Alice']
```


```
months1 = ["January", "February", "March"]
months2 = ["April", "May", "June"]
months1.extend(months2)
print(months1)
```

OUTPUT:

```
['January', 'February', 'March', 'April', 'May', 'June']
```

More methods to use with Lists

Method	Description
append()	Adds an element at the end of the list
clear()	Removes all the elements from the list
copy()	Returns a copy of the list
count()	Returns the number of elements with the specified value
extend()	Add the elements of a list (or any iterable), to the end of the current list
index()	Returns the index of the first element with the specified value
insert()	Adds an element at the specified position
pop()	Removes the element at the specified position
remove()	Removes the first item with the specified value
reverse()	Reverses the order of the list
sort()	Sorts the list

Source: https://www.w3schools.com/python/python_ref_list.asp

Looping through a list

You can loop through the list items by using a **for loop:**

```
fruits= ["apple", "banana", "cherry"]
for fruit in fruits: #loop a list
  print(fruit)
```

OUTPUT:

```
apple
banana
cherry
```


```
for number in range(2): #directly use range() to loop
  print(number)
```

OUTPUT:

```
0
1
```

Looping through a list (enumerate)

enumerate() is a function that is used with for loops to get both the index and the item of an iterable (like a list) at the same time.

Without enumerate:

```
fruits = ["apple", "banana", "cherry"]

i = 0
for fruit in fruits:
    print(i, fruit)
    i += 1
```

OUTPUT:

```
0 apple
1 banana
2 cherry
```

Looping through a list (enumerate)


```
fruits= ["apple", "banana", "cherry"]
#loop a list with enumerate
for index, fruit in enumerate(fruits):
  print(index, fruit, sep=": ")
```

OUTPUT:

```
0: apple
1: banana
2: cherry
```

How it works: enumerate(fruits) returns pairs of (index, value) for each item, the loop unpacks each pair into i and fruit automatically.

Starting the count from a different number:

```
for i, fruit in enumerate(fruits, start=1):
    print(i, fruit)
```

Randomization in list

Python **random module** provides several functions to accomplish this. Import it to use it.

- random.shuffle() = shuffle the list
- random.choice() = pick one from the list

```
import random
my_list = ['A', 'B', 'C', 'D', 'E']
random.shuffle(my_list)
print(my_list)

OUTPUT: ['D', 'E', 'A', 'B', 'C']
```


```
import random
fruits = ['apple', 'banana', 'cherry', 'grape']
random_fruit = random.choice(fruits)
print(random_fruit)

OUTPUT: cherry
```

List comprehension

List comprehension is a shorter and elegant way to create lists in Python.
It provides a shorter syntax:

Task: create a list of the squares of the first five numbers

```
#Traditional for loop way
numbers = range(1, 6) #[1, 2, 3, 4, 5]
squares = []
for number in numbers:
    squares.append(number ** 2)
print(squares)
```


```
#List comprehension way
numbers = range(1, 6) #[1, 2, 3, 4, 5]
squares = [num ** 2 for num in numbers]
print(squares)
OUTPUT: [1, 4, 9, 16, 25]
```

Quiz Time

Time Duration: 3 mins

#Add 1 to the following list **“x”**.
#(Please do it both ways - traditional and list comprehension way)
x = [1, 2, 3, 4, 5]
#Desired output: [2, 3, 4, 5, 6]

Quiz Time

Time Duration: 3 mins

#Add 1 to the following list **"x"**.
#(Please do it both ways - traditional and list comprehension way)
x = [1, 2, 3, 4, 5]
#Desired output: [2, 3, 4, 5, 6]

Solution (Traditional Way)

```
x = [1, 2, 3, 4, 5]
y = []
for item in x:
    y.append(item + 1)
print(y)
```

OUTPUT:

```
[2, 3, 4, 5, 6]
```

Solution (List Comprehension Way)

```
#List comprehension way
x = [1, 2, 3, 4, 5]
y = [item + 1 for item in x]
print(y)
```

OUTPUT:

```
[2, 3, 4, 5, 6]
```

Homework Name: Lab1HW2

1. String Slicing (1 Point)

Instructions: Declare a string variable and assign it the value "Python is so easy". Then, write code to extract and print the following parts:

- The first 6 characters ("Python").
- The last 4 characters ("easy").

2. Reverse a String (1 Point)

Instructions: Write a single line of code that reverses the string "TIAEVOLI" using slicing and prints the result.

3. List Comprehension (1 Point)

Instructions: Given the list **words** = ["code", "syntax", "function"], use a list comprehension to create a new list where each item has "python_" added to the front.

The final output should be ['python_code', 'python_syntax', 'python_function'].

Homework Name: Lab1HW2

4. Pairing Lists with zip() (2 Point)

Instructions:

- Create a list called **friends** that contains three of your friends' names.
- Create a second list called **colors** that contains their favorite colors, making sure the colors match the order of the names in the first list.
- Use a for loop along with the zip() method to iterate through both lists at the same time.
- Inside the loop, print each friend's name followed by their corresponding favorite color.

Example Output:
William White
Crystal Pink
Michael Black

Hint: The **zip() function** creates pairs from multiple lists.
Your for loop will need to use two variables to "unpack" these pairs, like this:
for friend, color in zip(friends, colors):

Reference on how to use: https://www.w3schools.com/python/ref_func_zip.asp

Homework Instructions

Submission Format:

- Please submit your assignments as either a **.ipynb** (Jupyter Notebook) file or a .**pdf** file.
- If you choose to submit a .**pdf**, make sure all of your code and its output are **clearly visible.**

File Naming Convention:

- You must name your file exactly as shown below:
- YourName_YourStudentID_AssignmentName.FileExtension
- **Example**: Anushka_st126222_Lab2HW.pdf

Late Submissions:

- Please submit all assignments **on time**. Late submissions will be **penalized**.
- Submissions made **within 24 hours after the deadline** will receive **50% penalty**.
- Submissions will **not be accepted** more than **24 hours after the deadline**.

Resource for Further Studies into Python

https://www.w3schools.com/python/

Any Questions?

Send Email at: st126222@ait.ac.th